



# Movie Rental Project

Prepared by:

Mo'men Mhana (12323848)

Samer Awaissah(12323992)

Supervisor: Dr. Mo'men Abughazala

An-Najah National University

Faculty of Information Technology and Artificial Intelligence

Department of Computer Science

# 1. Introduction

The purpose of this project is to design and implement a data warehouse for a movie rental business using the Sakila OLTP database as the source system. The OLTP database supports daily operations such as film rentals, payments, customer management, and inventory tracking. Although it is efficient for transactional processing, it is not optimized for analytical reporting and business analysis.

The proposed data warehouse reorganizes the operational data into a dimensional model based mainly on a Star Schema with a hybrid approach for handling the many-to-many relationship between films and categories through a bridge table. The warehouse includes several dimension and fact tables that support analysis of rental activity, revenue, customer behavior, film popularity, and store performance.

The ETL process was implemented using Python, Pandas, and SQLAlchemy to extract, transform, clean, and load the data into the warehouse. Data quality rules such as null handling, duplicate removal, and surrogate key validation were also applied to ensure reliable analytical data.

Unlike the OLTP system, which focuses on daily transactions, the data warehouse is designed to support analytical queries such as identifying the most rented films, analyzing revenue trends, and evaluating store and staff performance.

## 2. Business Questions

The proposed data warehouse is designed to support analytical reporting and help managers make better business decisions. The following business questions represent the main analytical areas supported by the warehouse.

| Business Question                                           | Importance                                                                            |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Which films are rented most frequently?                     | Helps identify the most popular films and supports inventory and marketing decisions. |
| Which films generate the highest revenue?                   | Helps determine the most profitable films and categories.                             |
| Which stores generate the highest number of rentals?        | Supports evaluation of store activity and customer demand across locations.           |
| Which stores generate the highest revenue?                  | Helps identify the most profitable stores and compare branch performance.             |
| Which customers generate the highest revenue?               | Identifies high-value customers and supports customer relationship management.        |
| How does rental activity change over time?                  | Helps analyze trends, seasonal demand, and business performance over time.            |
| Which staff members process the highest number of payments? | Supports employee performance evaluation and workload analysis.                       |
| Which films are returned late most often?                   | Helps analyze late return behavior and improve rental policies.                       |
| What is the average rental duration for films?              | Helps evaluate customer rental behavior and film usage patterns.                      |
| How does store performance vary by rentals and revenue?     | Supports overall store performance analysis using rentals, revenue, and late returns. |

## 3. Dimensional Model Design

### 3.1. Selected Business Processes

The main business processes identified from the OLTP system are:

1. Film Rental Transactions
2. Payment Transactions
3. Store Performance Analysis

These processes became the main fact tables in the data warehouse.

---

### 3.2. Fact Tables

#### Fact\_Rental\_Film

| Item               | Description                                                       |
|--------------------|-------------------------------------------------------------------|
| Business Process   | Film rental transactions                                          |
| Grain              | One record for each rental transaction                            |
| Measures           | rental_duration, days_late, rental_count                          |
| Related Dimensions | dim_film, dim_store, dim_date                                     |
| Source OLTP Tables | rental, inventory, film, store                                    |
| Purpose            | Used to analyze rental activity, late returns, and rental trends. |

---

#### Fact\_Payment

| Item               | Description                                                        |
|--------------------|--------------------------------------------------------------------|
| Business Process   | Payment transactions                                               |
| Grain              | One record for each payment transaction                            |
| Measures           | amount, payment_count                                              |
| Related Dimensions | dim_customer, dim_film, dim_store, dim_staff, dim_date             |
| Source OLTP Tables | payment, rental, inventory, film, customer, store, staff           |
| Purpose            | Used to analyze revenue by customer, film, store, staff, and time. |

---

### Fact\_Store\_Performance

| Item               | Description                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------------|
| Business Process   | Store performance analysis                                                                        |
| Grain              | One record per store per date                                                                     |
| Measures           | total_rentals, total_revenue, total_late_returns                                                  |
| Related Dimensions | dim_store, dim_location, dim_date                                                                 |
| Source OLTP Tables | rental, payment, inventory, store, address, city, country                                         |
| Purpose            | Used to analyze overall store performance, revenue trends, and late returns by location and time. |

## 3.3. Dimension Tables

### Dim\_Film

| Item               | Description                                               |
|--------------------|-----------------------------------------------------------|
| PK                 | film_key                                                  |
| Business Key       | film_id                                                   |
| Main Attributes    | title, rating, rental_rate, rental_duration, release_year |
| Source OLTP Tables | film                                                      |

---

### Dim\_Category

| Item               | Description   |
|--------------------|---------------|
| PK                 | category_key  |
| Business Key       | category_id   |
| Main Attributes    | category_name |
| Source OLTP Tables | category      |

---

### Dim\_Customer

| Item               | Description                      |
|--------------------|----------------------------------|
| PK                 | customer_key                     |
| Business Key       | customer_id                      |
| Main Attributes    | full_name, email, city, country  |
| Source OLTP Tables | customer, address, city, country |

---

### Dim\_Date

| Item               | Description                               |
|--------------------|-------------------------------------------|
| PK                 | date_key                                  |
| Main Attributes    | full_date, day_name, month, quarter, year |
| Source OLTP Tables | Derived from rental_date and payment_date |

---

### Dim\_Store

| Item               | Description                          |
|--------------------|--------------------------------------|
| PK                 | store_key                            |
| Business Key       | store_id                             |
| Main Attributes    | manager_name, city, country          |
| Source OLTP Tables | store, staff, address, city, country |

---

### Dim\_Staff

| Item               | Description         |
|--------------------|---------------------|
| PK                 | staff_key           |
| Business Key       | staff_id            |
| Main Attributes    | full_name, store_id |
| Source OLTP Tables | staff               |

---

## Dim\_Location

| Item               | Description             |
|--------------------|-------------------------|
| PK                 | location_key            |
| Main Attributes    | city, country, district |
| Source OLTP Tables | address, city, country  |

---

## Bridge Table: Bridge\_Film\_Category

| Attribute    | Description          | Key Type    |
|--------------|----------------------|-------------|
| film_key     | Related film key     | Foreign Key |
| category_key | Related category key | Foreign Key |

## Shared Dimensions

| Dimension    | Shared Between                                         |
|--------------|--------------------------------------------------------|
| dim_film     | fact_payment, fact_rental_film                         |
| dim_store    | fact_payment, fact_rental_film, fact_store_performance |
| dim_date     | fact_payment, fact_rental_film, fact_store_performance |
| dim_customer | fact_payment                                           |
| dim_staff    | fact_payment                                           |
| dim_location | fact_store_performance                                 |

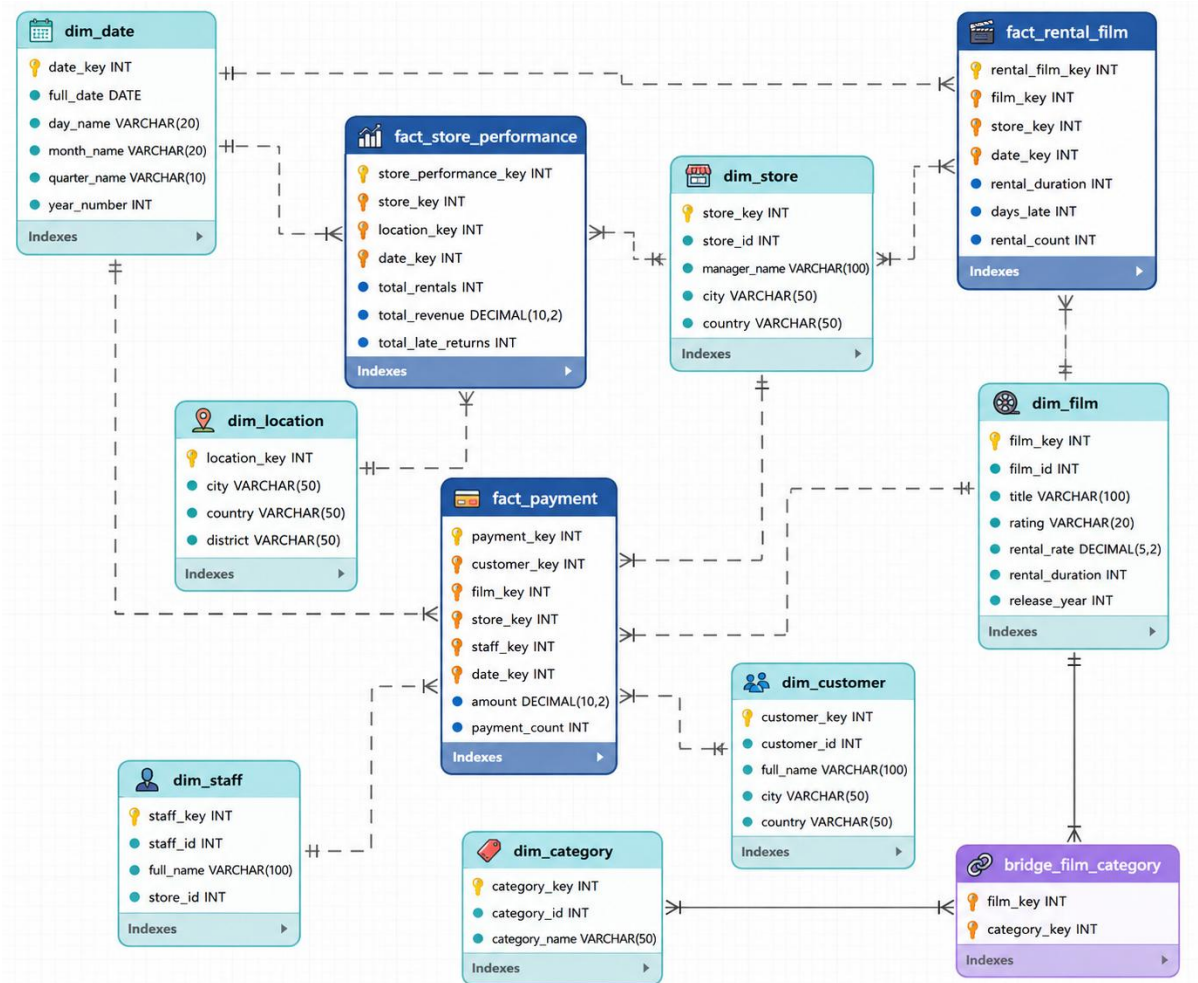
---

## Schema Type

The proposed design uses a Hybrid Schema:

- Most dimensions follow a Star Schema design.
- The relationship between films and categories is handled using a bridge table because one film may belong to multiple categories.
- The schema also includes a location dimension used in store performance analysis.

# Dimensional Model Diagram





## 5. ETL Design

The ETL (Extract, Transform, Load) process is responsible for transferring data from the operational OLTP system into the analytical data warehouse in a clean and structured format. In this project, the ETL pipeline was implemented using Python with the Pandas library and SQLAlchemy to connect to both the source Sakila database and the destination data warehouse database.

### 5.1. Extract

The first phase of the ETL process involves extracting raw data from the Sakila OLTP database into Pandas DataFrames. The following OLTP tables were extracted:

| OLTP Table    | Description                                                             |
|---------------|-------------------------------------------------------------------------|
| film          | Film information including title, rating, rental rate, and release year |
| category      | Film category information                                               |
| film_category | Junction table between films and categories                             |
| store         | Store information                                                       |
| staff         | Staff member information                                                |
| customer      | Customer information                                                    |
| rental        | Rental transaction records                                              |
| payment       | Payment transaction records                                             |
| inventory     | Links films to stores                                                   |
| address       | Address information                                                     |
| city          | City names linked to addresses                                          |
| country       | Country names linked to cities                                          |

All tables were extracted using the following pattern:

```
table = pd.read_sql("SELECT * FROM table_name", con=engine1)
```

The source OLTP database and the destination data warehouse database were connected using SQLAlchemy engine objects:

```
engine1 = create_engine(srcConnection_string) # OLTP source
```

```
engine2 = create_engine(dstConnection_string) # Data Warehouse destination
```

## 5.2. Transform

The transformation phase is where raw operational data is cleaned, reorganized, and prepared to fit the dimensional model of the data warehouse. The following transformations were applied for each dimension and fact table.

### 5.2.1. dim\_film

- The film table was transformed into a film dimension table.
  - Only analytical attributes were retained, including film\_id, title, rating, rental\_rate, rental\_duration, and release\_year.
  - Unnecessary operational columns were excluded.
- 

### 5.2.2. dim\_category

- Extracted directly from the category OLTP table with minimal transformation.
  - The column name was renamed from name to category\_name for clarity.
- 

### 5.2.3. bridge\_film\_category

- The film\_category junction table was used to handle the many-to-many relationship between films and categories.
- Business keys (film\_id, category\_id) were replaced with surrogate keys (film\_key, category\_key) by joining with the loaded dim\_film and dim\_category tables in the warehouse.

### 5.2.4. dim\_customer

- The customer table was joined with address, city, and country tables to create a complete customer dimension record.
  - First name and last name were combined into a single full\_name attribute.
  - Customer location attributes such as city and country were included in the dimension table.
  - Unnecessary operational columns were excluded.
-

### 5.2.5. dim\_store

- The store table was joined with staff, address, city, and country tables.
  - The manager's first and last names were combined into a single manager\_name attribute.
  - Store location attributes such as city and country were integrated into the dimension table.
- 

### 5.2.6. dim\_staff

- The staff table was transformed into a staff dimension table.
  - First and last names were concatenated into a full\_name attribute.
  - The related store\_id was retained for store association.
- 

### 5.2.7. dim\_location

- The location dimension was created by joining address, city, and country tables.
  - The final dimension includes city, country, and district attributes.
  - Duplicate location records were removed using drop\_duplicates().
- 

### 5.2.8. dim\_date

The date dimension was created by collecting date values from:

- rental\_date from the rental table
- payment\_date from the payment table

The following transformations were applied:

- Normalization: .dt.normalize() was used to remove the time component from datetime values.
- Null handling: NULL values were removed using dropna().
- Deduplication: Duplicate dates were removed using drop\_duplicates().
- Attribute derivation: Additional attributes were derived from each date, including: day\_name, month, quarter, year

### 5.2.9. fact\_rental\_film

- The rental table was joined with inventory to obtain film\_id and store\_id for each rental transaction.
- The film table was joined to retrieve the expected rental\_duration.
- The actual rental duration was calculated as the difference between return\_date and rental\_date in days.
- Late return days (days\_late) were calculated by subtracting the expected rental duration from the actual rental duration.
- If the calculated days\_late value was negative, it was replaced with 0 to indicate that the film was returned on time or early.
- Remaining NULL values in days\_late were filled with 0.
- Business keys (film\_id, store\_id, and rental date) were replaced with surrogate keys (film\_key, store\_key, and date\_key) by joining with the corresponding dimension tables in the warehouse.
- A rental\_count measure with a value of 1 was added to support rental aggregation and analytical reporting.

### 5.2.10. fact\_payment

- The payment table was joined with rental to obtain inventory\_id, and then joined with inventory to retrieve film\_id and store\_id.
- Surrogate keys were attached by joining with dim\_customer, dim\_film, dim\_store, dim\_staff, and dim\_date.
- The payment date was normalized using .dt.normalize() before joining with dim\_date to match only the date portion.
- The main measure retained in the fact table is amount, which represents the payment value.
- A payment\_count measure with a value of 1 was added to support payment aggregation and performance analysis.

## 5.3. Load

The load phase inserts the transformed data into the destination data warehouse database (`movie_rental`). Loading follows a specific order to maintain referential integrity and ensure that all foreign key relationships are valid.

### 5.3.1. Loading Order

Dimension tables were loaded before fact tables because fact tables depend on the surrogate keys generated in the dimensions. The loading order used in this project was:

1. `dim_film`
2. `dim_category`
3. `bridge_film_category`
4. `dim_store`
5. `dim_customer`
6. `dim_staff`
7. `dim_location`
8. `dim_date`
9. `fact_rental_film`
10. `fact_payment`
11. `fact_store_performance`

### 5.3.2. Surrogate Key Generation

Surrogate keys were generated automatically using the Pandas DataFrame index. Each dimension table was assigned an index named after its surrogate key (such as `film_key`, `customer_key`, and `date_key`), and the index was written to the warehouse tables using `index=True` in the `to_sql()` method.

```
dim_film.index.name = "film_key"
```

```
dim_film.to_sql(  
    "dim_film",  
    con=engine2,  
    if_exists="replace",  
    index=True)
```

This approach ensures that each dimension record receives a unique integer identifier independent of the original business keys.

### 5.3.3. Connecting Fact Tables to Dimensions

After loading the dimension tables, surrogate keys were retrieved from the warehouse and used to replace business keys in the fact tables.

```
dim_film_dw = pd.read_sql(  
    "SELECT film_id, film_key FROM dim_film",  
    con=engine2)
```

```
fact_rental_film = rf.merge(  
    dim_film_dw,  
    on="film_id",  
    how="left")
```

This process ensures that fact tables store surrogate keys instead of descriptive business identifiers. Only foreign keys and analytical measures were retained in the final fact tables before loading.

### 5.3.4. Handling Null Values Before Load

- NULL values in days\_late were replaced with 0 using fillna(0).
- Surrogate key columns were converted to integer type using astype(int) to maintain datatype consistency between fact and dimension tables

```
fact_rental_film = fact_rental_film.astype({  
    "film_key": int,  
    "store_key": int,  
    "date_key": int})
```

### 5.3.5. Load Method

All tables were loaded into the warehouse using the Pandas `to_sql()` method with `if_exists="replace"`, which recreates the tables during each ETL execution.

```
fact_rental_film.to_sql(  
    "fact_rental_film",  
    con=engine2,  
    if_exists="replace",  
    index=True)
```

This full-refresh loading approach ensures that the warehouse always reflects the latest data from the OLTP source system without keeping outdated or duplicate records.

## 6. Data Quality Considerations

Data quality is an important part of the ETL process because incomplete or inconsistent data can lead to incorrect analysis and unreliable business decisions. Several data quality checks and cleaning rules were applied during the ETL implementation to ensure accurate and consistent warehouse data.

### 1. Null Checks on Dimension Tables

After constructing the dimension tables, NULL values were checked using `isnull().sum()` to identify incomplete records before loading them into the warehouse. This validation was applied to tables such as `dim_film`, `dim_category`, `dim_store`, `dim_customer`, `dim_staff`, and `dim_location`.

```
print(dim_film.isnull().sum())  
print(dim_category.isnull().sum())  
print(dim_store.isnull().sum())  
print(dim_customer.isnull().sum())  
print(dim_staff.isnull().sum())  
print(dim_location.isnull().sum())
```

## 2. Null Detection and Removal from the Date Dimension

The `dim_date` table was created using date values collected from `rental_date` and `payment_date`. Before creating the dimension table, the ETL process checked for NULL values and removed them using `dropna()` to ensure that only valid dates were included in the warehouse.

```
print("null")

print(all_dates.isnull().sum())

all_dates.dropna(inplace=True)
```

## 3. Duplicate Detection and Removal from the Date Dimension

Since date values were collected from multiple operational tables, duplicate dates appeared frequently. The ETL process checked for duplicate records using `duplicated().sum()` and removed them using `drop_duplicates()` to ensure that each date appears only once in the `dim_date` table.

```
print("duplicated")

print(all_dates.duplicated().sum())

all_dates.drop_duplicates(inplace=True)

print(all_dates.duplicated().sum())

print(all_dates.isnull().sum())
```

## 4. Handling Negative days\_late Values

When films were returned before the expected rental duration, the calculated `days_late` value became negative. Since negative late days have no business meaning, the ETL process replaced all negative values with 0 using the Pandas `clip(lower=0)` function.

```
rf["days_late"] = (

    rf["actual_duration"] - rf["rental_duration"]

).clip(lower=0)
```



## 5. Handling NULL Values in days\_late

Some rental records may produce NULL values in days\_late, especially when return information is missing or incomplete. To ensure consistent analytical results and prevent NULL values from appearing in the fact table, the ETL process replaced remaining NULL values in days\_late with 0 using the Pandas fillna() function.

```
rf["days_late"] = rf["days_late"].fillna(0)
```

## 6. Null Checks on Fact Tables

After constructing the fact tables and attaching surrogate keys from the dimension tables, the ETL process checked for NULL values using isnull().sum(). This validation helps ensure that all joins were completed successfully and that each fact record is properly connected to its related dimensions.

```
print(fact_rental_film.isnull().sum())
```

```
print(fact_payment.isnull().sum())
```

```
print(fact_store_performance.isnull().sum())
```

## 7. Type Consistency for Surrogate Keys

Before loading the fact tables into the data warehouse, surrogate key columns were converted to integer type using astype(int). This ensures datatype consistency between foreign keys in fact tables and primary keys in dimension tables, helping maintain referential integrity.

```
fact_rental_film = fact_rental_film.astype({  
    "film_key": int,  
    "store_key": int,  
    "date_key": int})
```

## 7. Conclusion

This project successfully transformed the Sakila Movie Rental OLTP database into an analytical data warehouse using dimensional modeling and a Python-based ETL pipeline. The warehouse was designed mainly using a Star Schema structure with a hybrid approach for handling the many-to-many relationship between films and categories through a bridge table.

Several dimension tables and fact tables were created to support analytical reporting related to rental activity, revenue analysis, store performance, customer behavior, and late return analysis. The ETL process extracted data from the OLTP system, transformed and cleaned the data, generated surrogate keys, and loaded the final warehouse tables using Pandas and SQLAlchemy.

To improve data reliability, multiple data quality rules were applied during the ETL process, including NULL checks, duplicate removal, datatype consistency validation, and handling invalid `days_late` values. The warehouse also supported analytical visualizations such as top rented films, revenue trends, staff performance, and store performance analysis.

The final solution enables business users to answer important analytical questions more efficiently than the original OLTP system. Overall, the project demonstrates the practical implementation of data warehouse design, ETL development, data quality management, and business analytics using a realistic movie rental scenario.